

2022 学年第 2 学期《编译原理》(B 卷) 参考答案与解析

第一部分：参考答案

一、判断题（共 10 小题，每小题 1 分，共 10 分）

1. × (错误)。DFA 虽然不允许 ε 变迁，但只要初始状态同时也是接受状态，即可识别空串 ε 。识别 ε 不需要 ε 变迁。
2. ✓ (正确)。通过子集构造法 (Subset Construction)，任意 NFA 都可构造一个等价的 DFA。
3. × (错误)。文法 $A \rightarrow Ba \mid a$, $B \rightarrow A$ 存在间接左递归: $A \Rightarrow Ba \Rightarrow Aa$ 。
4. × (错误)。分析树的叶节点可能为 ε (空串)， ε 不是终结符。当文法含 ε -产生式时叶节点可为 ε 。
5. ✓ (正确)。LL(0) 意味着不向前看任何符号，每个非终结符至多只有一个候选产生式，因此最多产生一个句子。
6. × (错误)。LALR(1) 合并同心项集可能产生新的归约-归约冲突，但不会产生新的移进-归约冲突。
7. × (错误)。L-属性定义也可在自底向上的 LR 分析中实现，可通过扩充分析栈来支持。
8. ✓ (正确)。回填 (Backpatching) 正是用于解决一遍扫描时跳转目标尚未确定的问题。
9. × (错误)。全局优化指在一个过程内部跨基本块的优化；跨多个子程序的优化称为过程间优化 (Inter-procedural Optimization)。
10. × (错误)。指令选择、指令调度、寄存器分配属于目标代码生成阶段，而非中间代码优化阶段。

二、填空题（共 7 小题 10 个空，每空 1 分，共 10 分）

1. 上下文有关文法 / 1 型文法 (Type 1, CSG)
2. ε , a, aa, aac, aacB (含句柄非全长前缀或句柄正在形成的前缀均可)
3. 第一空: $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$; 第二空: $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$
4. 第一空: $n + 1$ (n 个终结符 + $\$$); 第二空: m (m 个非终结符, 不含 S')
5. 第一空: B 为真时控制流转向的代码标号 (即 S_1 第一条指令的标号); 第二空: S 执行完毕后下一条指令的标号
6. 引用计数法 (Reference Counting)
7. 6 (LD 读内存 2 + ADD 含内存操作数 2 + ST 写内存 2)

三、简答题（简要答案）

第 1 题

有二义性。句子 $\text{id} - \text{id} - \text{id}$ 可构造两棵不同的分析树: $(\text{id} - \text{id}) - \text{id}$ 与 $\text{id} - (\text{id} - \text{id})$ 。

第 2 题

2.1 $T.type$ 为综合属性; $L.type$ 为继承属性。

2.2 带注释分析树 (略, 见详细解答)。

第 3 题

基本块划分:

- $B_1 = \{1, 2\}$ (入口-出口型)
- $B_2 = \{3, 4\}$ (条件跳转)
- $B_3 = \{5, 6, 7\}$ (无条件跳转)
- $B_4 = \{8, 9\}$ (程序终止)

流图: $B_1 \rightarrow B_2, B_2 \rightarrow B_4, B_2 \rightarrow B_3, B_3 \rightarrow B_2$ 。

第 4 题

利用 DAG 消除公共子表达式和死代码后的优化结果：

```
S3 := X - Y
S4 := X + Y
A  := 4 * S4
B  := S3 * S4
```

四、应用题（简要答案）

第 1 题

1.1 $b^*(ab^*ab^*ab^*)^*b^*$ 或 $(b^*ab^*ab^*a)^*b^*$

1.2 DFA 含 3 个状态: q_0 (初始且接受), q_1 , q_2 。转换: $q_0 \xrightarrow{a} q_1$, $q_0 \xrightarrow{b} q_0$; $q_1 \xrightarrow{a} q_2$, $q_1 \xrightarrow{b} q_1$; $q_2 \xrightarrow{a} q_0$, $q_2 \xrightarrow{b} q_2$ 。

第 2 题

2.2 G1 含左递归 $A \rightarrow Ab$, 不适合自顶向下分析。G2: $S \rightarrow ABa$, $A \rightarrow bA \mid \varepsilon$, $B \rightarrow dB \mid c$ 。

2.3 FIRST/FOLLOW 见详细解答。

2.4 G2 为 LL(1) 文法。

第 3 题

3.1 G3: (0) $S' \rightarrow S$, (1) $S \rightarrow E + T$, (2) $E \rightarrow (E)$, (3) $E \rightarrow a$, (4) $T \rightarrow b$ 。

3.5 LR(0) 存在移进-归约冲突; SLR(1) 可解决 (因 $\text{FOLLOW}(T) = \{\$, \}$, 与移进符号不相交)。

3.6 LR(1) 为每个项目附加前瞻符, 比 SLR(1) 的 FOLLOW 集更精确。

第二部分：详细解答与推导过程

1 一、判断题详解

第 1 题

答案：错误。

DFA 的定义中确实不允许 ε 变迁，且每个状态对每个输入符号有且仅有一条转换。但这并不意味着 DFA 不能识别空串 ε 。只要 DFA 的初始状态 q_0 同时也是接受状态 ($q_0 \in F$)，则该 DFA 接受 ε 。因为从初始状态出发，不消耗任何输入符号，已处于接受状态。这是 FA 识别空串的标准方式，与 ε 变迁无关。

参考课件：“DFA: One transition per input per state, No ε -moves”——课件强调了 DFA 无 ε 变迁，但也指出接受状态可以直接是初始状态。

第 2 题

答案：正确。

这是自动机理论的基本定理之一。通过**子集构造法** (Subset Construction Algorithm)，可以将任意 NFA 转换为等价的 DFA。基本思想是让 DFA 的每个状态对应 NFA 的一个状态子集。

具体算法步骤：

- (1) 计算初始状态的 ε -闭包，作为 DFA 的初始状态
- (2) 对每个未标记的 DFA 状态（对应 NFA 状态子集 T ），对每个输入符号 a ，计算 $\varepsilon\text{-closure}(\text{move}(T, a))$
- (3) 若得到新的状态子集，将其加入 DFA 状态集合
- (4) 重复直到没有新状态产生

参考课件：“NFA and DFA are equivalent. To show this we must prove every DFA can be converted into an NFA which accepts the same language, and vice-versa.”

第 3 题

答案：错误。

考察文法的左递归：

$$A \rightarrow Ba \mid a, \quad B \rightarrow A$$

推导: $A \Rightarrow Ba \Rightarrow Aa$, 即 $A \Rightarrow^+ Aa$, 存在**间接左递归**。间接左递归同样是左递归的一种形式, 会使自顶向下分析陷入无限循环。

第 4 题

答案: 错误。

上下文无关文法的分析树 (Parse Tree) 定义中:

- 根节点标记为开始符号
- 每个内部节点标记为一个非终结符
- 每个叶节点标记为**终结符或 ε**

当文法中存在 ε -产生式 (如 $A \rightarrow \varepsilon$) 时, 对应的叶节点为 ε , 而 ε 不是终结符 (它是空串, 并非字母表中的符号)。因此原命题 "叶节点均为终结符" 不准确。

参考: 龙书 (Dragon Book) 定义: "The leaves of a parse tree are labeled by terminals or ε ."

第 5 题

答案: 正确。

LL(k) 中的 k 表示向前看 k 个符号 (lookahead)。 $k = 0$ 意味着完全不向前看任何输入符号。对于 LL(0) 文法, 在每个推导步骤中, 必须仅根据当前栈顶的非终结符 (不参考任何输入符号) 唯一确定使用哪个产生式。这意味着:

- 每个非终结符至多只有一个候选产生式
- 所有推导只有唯一的路径
- 因此最多只能产生一个句子 (甚至可能产生 0 个句子)

第 6 题

答案: 错误。

LALR(1) 通过合并 LR(1) 的**同心项集** (具有相同核心项但不同前瞻符的状态) 来减少状态数量。

关键性质:

- 合并**可能**产生新的**归约-归约冲突**。因为原来在不同前瞻符下的归约动作, 合并后可能在相同输入符号上产生多个归约选择。

- 合并**不可能**产生新的**移进-归约冲突**。移进动作仅取决于核心项（不依赖前瞻符），如果合并前没有移进-归约冲突，合并后也不会出现。

参考课件：“LALR(1) may introduce Reduce-Reduce conflicts, but NOT Shift-Reduce conflicts.”

第 7 题

答案：错误。

L-属性定义（L-Attributed Definition）的特点是：继承属性只依赖于**左兄弟节点或父节点**的属性，不依赖于右兄弟。

L-属性定义确实**适合**自顶向下的递归下降预测分析（因为遍历顺序与依赖方向一致），但**也可以在自底向上的 LR 分析中实现**。在 LR 分析中，可以通过扩充分析栈（同时存储符号和属性值）来支持 L-属性定义的求值。

第 8 题

答案：正确。

回填（Backpatching）是语法制导翻译中用于**一遍扫描**生成中间代码的关键技术。当生成条件跳转或无条件跳转指令时，跳转的目标标号可能尚未定义（前向跳转）。回填的做法是：

- (1) 先生成一条带有空目标地址的跳转指令
- (2) 将该指令的索引记录在待回填列表中
- (3) 当目标标号最终被定义（生成对应指令）时，将目标地址填入之前记录的所有跳转指令中

第 9 题

答案：错误。

在编译优化的分类中：

- **局部优化（Local Optimization）**：在单个**基本块**范围内进行优化
- **全局优化（Global Optimization）**：在一个**过程/函数内部**，跨多个基本块进行优化（intra-procedural）

- **过程间优化 (Inter-procedural Optimization, IPO):** 跨多个子程序 (函数、过程等) 进行优化

题目描述的是过程间优化, 而非全局优化。

第 10 题

答案: 错误。

编译各阶段关注的问题不同:

- **中间代码优化:** 关注**机器无关**的优化, 如公共子表达式消除 (CSE)、死代码消除 (DCE)、常量传播、循环优化等
- **目标代码生成:** 关注**机器相关**的问题, 如指令选择 (Instruction Selection)、指令调度 (Instruction Scheduling)、寄存器分配与指派 (Register Allocation and Assignment)

2 二、填空题详解

第 1 题: Chomsky 文法分类

文法:

$$S \rightarrow aAC \mid aSAC$$

$$aA \rightarrow ab$$

$$bA \rightarrow bb$$

$$bC \rightarrow bc$$

$$cC \rightarrow cc$$

分析: 产生式 $aA \rightarrow ab$ 、 $bA \rightarrow bb$ 、 $bC \rightarrow bc$ 、 $cC \rightarrow cc$ 的左部均含有**两个符号** (一个终结符 + 一个非终结符), 且左部长度不超过右部长度 (均为 2), 右部非空。这符合 **1 型文法** (上下文有关文法, CSG) 的定义: 产生式形如 $\alpha A \beta \rightarrow \alpha \gamma \beta$ ($|\gamma| \geq 1$), 其中 A 的替换依赖于其上下文。

答案: 上下文有关文法 (1 型文法/CSG)

第 2 题: 活前缀

已知句型 $aacBb$, 句柄为 cB 。

活前缀定义为：句型中不包含句柄右侧任何符号的前缀。换言之，活前缀是句型的一个前缀，其在分析栈中不会越过句柄的右端。

对于句型 $aacBb$ （句柄为 cB ），活前缀包括：

$$\varepsilon, a, aa, aac, aacB$$

其中 $aacB$ 是包含了完整句柄的活前缀（表示即将归约）， ε, a, aa, aac 是不含或部分包含句柄的活前缀。

答案： $\varepsilon, a, aa, aac, aacB$

第 3 题：LL(1) 文法的充要条件

若文法 G 是 LL(1) 的，则对任意两个不同的产生式 $A \rightarrow \alpha \mid \beta$ 须满足：

第一空： $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$

即两个候选式的 FIRST 集不相交，确保向前看一个符号即可确定使用哪个产生式。

第二空（若 $\varepsilon \in \text{FIRST}(\alpha)$ ）： $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$

若 α 可推导出 ε ，则选择 $A \rightarrow \alpha$ 的条件也包括当前输入符号在 $\text{FOLLOW}(A)$ 中。为不产生冲突， $\text{FIRST}(\beta)$ 与 $\text{FOLLOW}(A)$ 必须不相交。

第 4 题：SLR(1) 分析表的列数

给定文法 G 有 m 个非终结符和 n 个终结符：

- **ACTION 表列数：** $n + 1$ 。 n 列对应 n 个终结符，额外 1 列对应结束符 $\$$ 。
- **GOTO 表列数：** m 。每列对应原始文法的一个非终结符（拓广文法引入的 S' 通常不出现在 GOTO 表的列中）。

第 5 题：语法制导翻译中的属性

对于产生式 $S \rightarrow \text{if } (B) S_1$ ：

- $B.\text{true}$ ：表示布尔表达式 B 的**真出口**——即 B 为真时控制流应转向的代码标号（通常是 S_1 第一条指令的标号）
- $S.\text{next}$ ：表示语句 S 执行完毕后下一条应执行指令的标号（即紧跟 S 代码之后的指令标号）

第 6 题：垃圾回收

常见的垃圾回收算法中，引用计数法 (Reference Counting) 无法回收循环引用的垃圾对象。因为循环引用的对象互相持有对方的引用，每个对象的引用计数至少为 1，永远不会降为 0，导致无法被回收。而标记-清除 (Mark-Sweep)、复制 (Copying)、分代 (Generational) 等追踪式 GC 算法可以正确处理循环引用。

第 7 题：目标代码开销

指令序列：

```
LD  R0, b
ADD R0, R0, c
ST  a, R0
```

开销分析：目标代码的指令开销需考虑内存访问代价。通常每条指令基值计 1 单位，每涉及一次内存访问（读或写）再加 1 单位：

- LD R0, b: 指令基值 1 + 读内存 b 的访问代价 1 = 2
- ADD R0, R0, c: 指令基值 1 + 读内存 c 的访问代价 1 = 2
- ST a, R0: 指令基值 1 + 写内存 a 的访问代价 1 = 2

共计 6 单位开销。

3 三、简答题详解

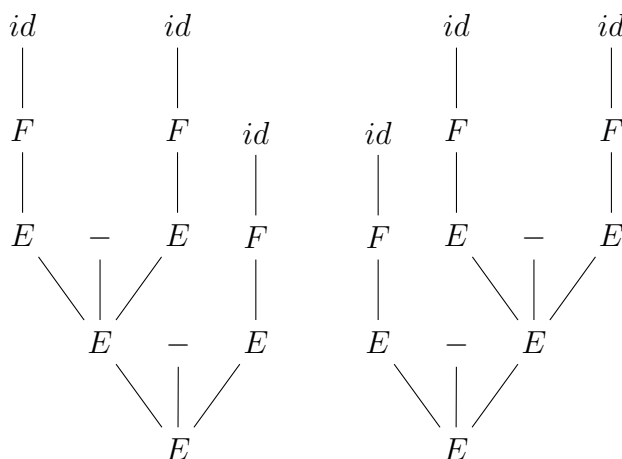
3.1 第 1 题 (5 分)：文法的二义性

文法 G ：

$$E \rightarrow E - E \mid F, \quad F \rightarrow id$$

答：该文法有二义性。

证明：以句子 $id - id - id$ 为例，可构造两棵不同的分析树：



左图对应 $(id - id) - id$ (左结合), 右图对应 $id - (id - id)$ (右结合)。它们对应不同的语义值, 说明文法具有二义性。二义性根因在于运算符 $-$ 的结合性未在文法中规定。

3.2 第 2 题 (5 分): 语法制导翻译

2.1 属性类型判断 (2 分)

翻译模式:

- $$\begin{array}{ll}
(1) D \rightarrow T L & \{L.type = T.type\} \\
(2) T \rightarrow \mathbf{int} & \{T.type = int\} \\
(3) T \rightarrow \mathbf{float} & \{T.type = float\} \\
(4) L \rightarrow L_1, id & \{L_1.type = L.type; \text{ addtype}(id.entry, L.type)\} \\
(5) L \rightarrow id & \{\text{ addtype}(id.entry, L.type)\}
\end{array}$$

- **$T.type$ 是综合属性 (Synthesized Attribute):** 由产生式 (2)(3) 从产生式右部直接计算得出 (int 或 float), 从子节点向父节点传递。
- **$L.type$ 是继承属性 (Inherited Attribute):** 由产生式 (1) 从 $T.type$ 传递给 $L.type$, 并从父节点 L 经产生式 (4) 传递给子节点 L_1 。继承属性的计算依赖于父节点或左兄弟的信息。

2.2 带注释分析树 (3 分)

对声明语句 `int y, z;`, 构造带注释的分析树:



```

      T    L (type=int)
    /      /\
  int    L , id(z)  { addtype(z.entry, int) }
(type    |
=int)    id(y)      { addtype(y.entry, int) }

```

属性求值过程：

- (1) $T.type = int$ (由 $T \rightarrow \mathbf{int}$ 计算)
- (2) $L.type = T.type = int$ (由 $D \rightarrow TL$ 传递)
- (3) 在子树 $L \rightarrow L_1, id$ 中, $L_1.type = L.type = int$, 并调用 $addtype(z.entry, int)$
- (4) 在子树 $L \rightarrow id$ 中, 调用 $addtype(y.entry, int)$

3.3 第 3 题 (5 分): 基本块划分与流图

基本块划分

中间代码：

```

1: load A
2: load B
3: L0: C := A mul B
4: if C > 56 goto L1
5: A := A + 1
6: B := B + 1
7: goto L0
8: L1: store C
9: halt

```

根据基本块的划分规则（入口：程序首条指令 / 跳转目标；出口：跳转指令 / 下条是跳转目标的指令 / halt）：

- $B_1 = \{1, 2\}$ (入口：程序首条指令；出口：语句 3 是跳转目标)
- $B_2 = \{3, 4\}$ (入口：标号 L_0 ；出口：条件跳转指令)
- $B_3 = \{5, 6, 7\}$ (入口：紧接条件跳转的下一条；出口：无条件跳转)
- $B_4 = \{8, 9\}$ (入口：标号 L_1 ；出口：halt 终止)

流图

$$B_1 \longrightarrow B_2, \quad B_2 \xrightarrow{\text{条件为真}} B_4, \quad B_2 \xrightarrow{\text{条件为假}} B_3, \quad B_3 \longrightarrow B_2$$

3.4 第 4 题 (5 分): DAG 优化基本块

原始代码:

```

1:  S1 := 10
2:  S2 := 40 / S1          // S2 = 4
3:  S3 := X - Y
4:  S4 := X + Y
5:  A  := S2 * S4          // A = 4*(X+Y)
6:  B  := A                // 死代码: B随后被重新定义
7:  S5 := X + Y            // 公共子表达式, 与S4相同
8:  S6 := S3 * S5          // S6 = (X-Y)*(X+Y)
9:  B  := S6               // 最终B = (X-Y)*(X+Y)

```

分析:

- $S_1 = 10$, $S_2 = 40/S_1 = 4$ 是常量, 可常量折叠
- $S_4 = X + Y$, $S_5 = X + Y$ 是公共子表达式, 可消除 S_5
- $S_6 \leftarrow S_3 * S_4$ (利用已消除的公共子表达式)
- 指令 6 ($B \leftarrow A$) 是死代码, 因为 B 在指令 9 被重新定义
- 仅 A 和 B 在基本块出口活跃

优化后基本块:

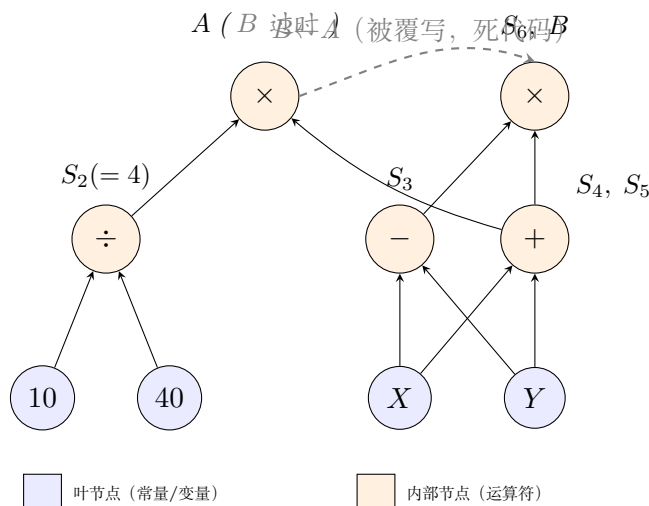
```

S3 := X - Y
S4 := X + Y
A  := 4 * S4
B  := S3 * S4

```

说明: 常量 $S_2 = 4$ 直接内联; 死赋值 $B \leftarrow A$ 删除; 公共子表达式 S_5 消除。

DAG 图示:



DAG 优化说明:

- **常量折叠:** 10 和 40 在 $\div$ 节点处被折叠为常量 4 (S_2), 消去了 S_1
- **公共子表达式消除:** $X + Y$ 的 $+$ 节点同时标注了 S_4 和 S_5 , 表示仅计算一次, S_5 被消除
- **死代码消除:** A 节点上的 B 标注被画为灰色—— $B \leftarrow A$ (指令 6) 随后被 $B \leftarrow S_6$ (指令 9) 覆写, 这条赋值是死代码
- **出口活跃变量:** 仅 A ($\times$ 节点左) 和 B ($\times$ 节点右) 在基本块出口处活跃, 其余变量均为临时

4 四、应用题详解

4.1 第 1 题 (15 分): 正则表达式与 DFA

1.1 正则表达式 (8 分)

字母表 $\Sigma = \{a, b\}$, 要求每个句子中 a 的个数是 3 的倍数 (0, 3, 6, 9, ...).

核心思想: 在任意多个 b 之间穿插 a , 且 a 的个数为 3 的倍数。即 a 必须以 3 个为一组出现。

正则表达式:

$$b^* (a b^* a b^* a b^*)^* b^*$$

或等价形式:

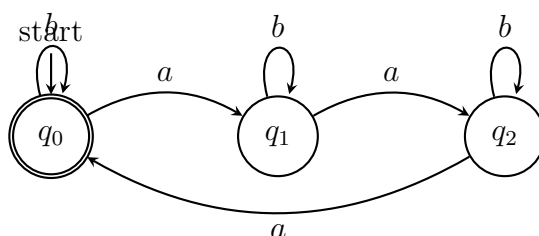
$$(b^* a b^* a b^* a)^* b^*$$

解释： (ab^*ab^*a) 保证每组恰好 3 个 a （中间可穿插任意多个 b ）， $*$ 表示任意多组，两侧 b^* 允许任意位置出现任意多个 b 。

1.2 DFA 设计 (7 分)

设计思路： 用 3 个状态记录已读取的 a 的个数模 3 的值。

- q_0 （初始且接受）：已读 a 的个数 $\equiv 0 \pmod{3}$
- q_1 ：已读 a 的个数 $\equiv 1 \pmod{3}$
- q_2 ：已读 a 的个数 $\equiv 2 \pmod{3}$



转换函数表：

状态	a	b
$\rightarrow^* q_0$	q_1	q_0
q_1	q_2	q_1
q_2	q_0	q_2

验证：对于任意输入串， $q_0 \xrightarrow{*} q_i$ 当且仅当其中 a 的个数 $\equiv i \pmod{3}$ 。仅当最终状态为 q_0 时接受。

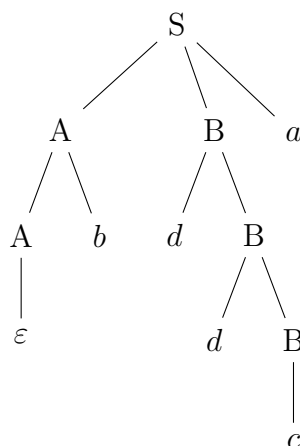
4.2 第 2 题 (20 分)：LL(1) 分析

文法 G1:

$$\begin{aligned}
 S &\rightarrow ABa \\
 A &\rightarrow Ab \mid \varepsilon \\
 B &\rightarrow dB \mid c
 \end{aligned}$$

2.1 句子 $bddca$ 的分析树 (2 分)

推导过程： $S \Rightarrow ABa \Rightarrow AbBa \Rightarrow bBba \Rightarrow bdBba \Rightarrow bddBba \Rightarrow bddca$



2.2 改写为 G2 (2 分)

G1 中 $A \rightarrow Ab$ 存在直接左递归，不适合自顶向下分析。

消除左递归： $A \rightarrow Ab \mid \varepsilon$ 等价于 $A \rightarrow bA \mid \varepsilon$ (这两者都产生语言 $\{b^n \mid n \geq 0\}$)。

等价文法 G2:

- (1) $S \rightarrow ABa$
- (2) $A \rightarrow bA$
- (3) $A \rightarrow \varepsilon$
- (4) $B \rightarrow dB$
- (5) $B \rightarrow c$

2.3 FIRST 与 FOLLOW 集 (4 分)

FIRST 集:

$$\text{FIRST}(A) = \{b, \varepsilon\} \quad (\text{由 } A \rightarrow bA \text{ 得 } b, \text{ 由 } A \rightarrow \varepsilon \text{ 得 } \varepsilon)$$

$$\text{FIRST}(B) = \{d, c\} \quad (\text{由 } B \rightarrow dB \text{ 和 } B \rightarrow c)$$

$$\begin{aligned} \text{FIRST}(S) &= (\text{FIRST}(A) \setminus \{\varepsilon\}) \cup \text{FIRST}(B) \\ &= \{b\} \cup \{d, c\} = \{b, d, c\} \end{aligned}$$

FOLLOW 集: (设 \$ 为结束符)

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(A) = \text{FIRST}(Ba) \setminus \{\varepsilon\} = \{d, c\}$$

$$\text{FOLLOW}(B) = \{a\}$$

其中 $\text{FOLLOW}(A) = \{d, c\}$ 是因为在 $S \rightarrow ABa$ 中 A 后跟 B ，而 B 不能推导出

ε ($B \rightarrow dB \mid c$ 均推导出至少一个终结符), 故 $\text{FIRST}(Ba) = \text{FIRST}(B) = \{d, c\}$ 。
 $\text{FOLLOW}(B) = \{a\}$ 是因为在 $S \rightarrow ABa$ 中 B 后跟 a 。□

非终结符	FIRST	FOLLOW
S	$\{b, d, c\}$	$\{\$ \}$
A	$\{b, \varepsilon\}$	$\{d, c\}$
B	$\{d, c\}$	$\{a\}$

2.4 LL(1) 分析表 (6 分)

对于每个产生式:

- 对于 $A \rightarrow \alpha$, 将产生式填入表中 $[A, a]$ 位置, $\forall a \in \text{FIRST}(\alpha)$
- 若 $\varepsilon \in \text{FIRST}(\alpha)$, 则 $\forall b \in \text{FOLLOW}(A)$, 在 $[A, b]$ 处填入

LL(1) 分析表:

	a	b	c	d	$\$$
S		$S \rightarrow ABa$	$S \rightarrow ABa$	$S \rightarrow ABa$	
A		$A \rightarrow bA$	$A \rightarrow \varepsilon$	$A \rightarrow \varepsilon$	
B			$B \rightarrow c$	$B \rightarrow dB$	

判断: G2 是 LL(1) 文法。

原因: 分析表中每个格子至多只有一个产生式, 无冲突:

- S : b, c, d 列各有唯一产生式, 无冲突
- A : b 列为 $A \rightarrow bA$, c, d 列为 $A \rightarrow \varepsilon$, $\text{FIRST}(bA) \cap \text{FOLLOW}(A) = \{b\} \cap \{d, c\} = \emptyset$, 无冲突
- B : c 列为 $B \rightarrow c$, d 列为 $B \rightarrow dB$, $\text{FIRST}(c) \cap \text{FIRST}(dB) = \{c\} \cap \{d\} = \emptyset$, 无冲突

2.5 输入串 $bddca$ 的 LL(1) 分析过程 (6 分)

约定: 栈顶在左侧, 推导时弹出栈顶非终结符, 将产生式右部**逆序**压入 (最左符号在栈顶)。

Step #	Stack (top left)	Input	Action
0	$S\$$	$bddca\$$	—
1	$ABa\$$	$bddca\$$	$S \rightarrow ABa$
2	$bABa\$$	$bddca\$$	$A \rightarrow bA$
3	$ABa\$$	$ddca\$$	match b
4	$Ba\$$	$ddca\$$	$A \rightarrow \epsilon$
5	$dBa\$$	$ddca\$$	$B \rightarrow dB$
6	$Ba\$$	$dca\$$	match d
7	$dBa\$$	$dca\$$	$B \rightarrow dB$
8	$Ba\$$	$ca\$$	match d
9	$ca\$$	$ca\$$	$B \rightarrow c$
10	$a\$$	$a\$$	match c
11	$\$$	$\$$	match a / ACCEPT

分析成功，句子 $bddca$ 被接受。□

4.3 第 3 题 (25 分): LR 分析

已知文法:

$$\begin{array}{lll}
 \text{G1: } S \rightarrow E + T & E \rightarrow (E) \mid a & T \rightarrow b \\
 \text{G2: } S \rightarrow E + T & E \rightarrow (E) \mid a & T \rightarrow b \mid bE
 \end{array}$$

3.1 G1 的拓广文法 G3 (1 分)

引入新的开始符号 S' 和产生式 $S' \rightarrow S$:

$$\begin{array}{l}
 (0) S' \rightarrow S \\
 (1) S \rightarrow E + T \\
 (2) E \rightarrow (E) \\
 (3) E \rightarrow a \\
 (4) T \rightarrow b
 \end{array}$$

3.2 LR(0) 项目集规范族 (活前缀 DFA) (6 分)

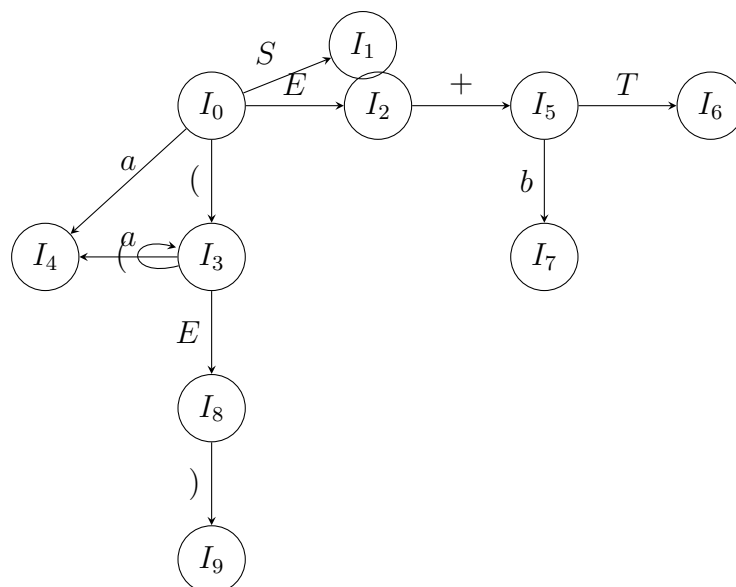
约定: 大写字母为非终结符, $\text{goto}(I_i, X) = I_j$ 表示从状态 I_i 经符号 X 到达状态 I_j 。

$$\begin{aligned}
I_0 &= \text{closure}(\{[S' \rightarrow \bullet S]\}) \\
&= \{S' \rightarrow \bullet S, S \rightarrow \bullet E + T, E \rightarrow \bullet(E), E \rightarrow \bullet a\} \\
I_1 &= \text{goto}(I_0, S) = \{S' \rightarrow S \bullet\} \quad (\text{接受状态}) \\
I_2 &= \text{goto}(I_0, E) = \{S \rightarrow E \bullet + T\} \\
I_3 &= \text{goto}(I_0, () = \{E \rightarrow (\bullet E), E \rightarrow \bullet(E), E \rightarrow \bullet a\} \\
I_4 &= \text{goto}(I_0, a) = \{E \rightarrow a \bullet\} \quad (\text{归约状态, 用产生式 (3)}) \\
I_5 &= \text{goto}(I_2, +) = \{S \rightarrow E + \bullet T, T \rightarrow \bullet b\} \\
I_6 &= \text{goto}(I_5, T) = \{S \rightarrow E + T \bullet\} \quad (\text{归约状态, 用产生式 (1)}) \\
I_7 &= \text{goto}(I_5, b) = \{T \rightarrow b \bullet\} \quad (\text{归约状态, 用产生式 (4)}) \\
I_8 &= \text{goto}(I_3, E) = \{E \rightarrow (E \bullet)\} \\
I_9 &= \text{goto}(I_8,) = \{E \rightarrow (E) \bullet\} \quad (\text{归约状态, 用产生式 (2)})
\end{aligned}$$

状态转移关系:

$$\begin{aligned}
I_0 &\xrightarrow{S} I_1, I_0 \xrightarrow{E} I_2, I_0 \xrightarrow{(} I_3, I_0 \xrightarrow{a} I_4 \\
I_2 &\xrightarrow{+} I_5 \\
I_3 &\xrightarrow{E} I_8, I_3 \xrightarrow{(} I_3, I_3 \xrightarrow{a} I_4 \\
I_5 &\xrightarrow{T} I_6, I_5 \xrightarrow{b} I_7 \\
I_8 &\xrightarrow{)} I_9
\end{aligned}$$

活前缀 DFA 图示:



3.3 LR(0) 分析表 (5 分)

State	ACTION						GOTO		
	<i>a</i>	<i>b</i>	(	)	+	\$	<i>S</i>	<i>E</i>	<i>T</i>
0	<i>s4</i>		<i>s3</i>				1	2	
1						<i>acc</i>			
2					<i>s5</i>				
3	<i>s4</i>		<i>s3</i>					8	
4	<i>r3</i>	<i>r3</i>	<i>r3</i>	<i>r3</i>	<i>r3</i>	<i>r3</i>			
5		<i>s7</i>							6
6	<i>r1</i>	<i>r1</i>	<i>r1</i>	<i>r1</i>	<i>r1</i>	<i>r1</i>			
7	<i>r4</i>	<i>r4</i>	<i>r4</i>	<i>r4</i>	<i>r4</i>	<i>r4</i>			
8				<i>s9</i>					
9	<i>r2</i>	<i>r2</i>	<i>r2</i>	<i>r2</i>	<i>r2</i>	<i>r2</i>			

说明: sX 表示移进到状态 X , rX 表示按产生式 X 归约, gX 表示 GOTO 状态 X 。

3.4 输入串 $(a) + b$ 的 LR 分析过程 (5 分)

约定: 符号栈和状态栈各自维护, GOTO 只在归约后使用。

Step	Stack (Symbols / States)	Input	Action
0	\$ 0	$(a) + b \$$	shift (, goto 3
1	\$ (0 3	$a) + b \$$	shift <i>a</i> , goto 4
2	\$ (a 0 3 4	$) + b \$$	reduce by (3) $E \rightarrow a$
3	\$ (E 0 3 8	$) + b \$$	shift), goto 9
4	\$ (E) 0 3 8 9	$+b \$$	reduce by (2) $E \rightarrow (E)$
5	\$ E 0 2	$+b \$$	shift +, goto 5
6	\$ E + 0 2 5	$b \$$	shift <i>b</i> , goto 7
7	\$ E + b 0 2 5 7	\$	reduce by (4) $T \rightarrow b$
8	\$ E + T 0 2 5 6	\$	reduce by (1) $S \rightarrow E + T$
9	\$ S 0 1	\$	ACCEPT

分析成功, 输入串 $(a) + b$ 被接受。□

Step 3 详解 (归约 $E \rightarrow a$): 状态 4 为 $\{E \rightarrow a\bullet\}$, 执行归约: 弹出 1 个符号 (a) 和 1 个状态 (4), 栈顶变为状态 3。goto(I_3, E) = I_8 , 压入 E 和状态 8。

Step 4 详解 (归约 $E \rightarrow (E)$): 状态 9 为 $\{E \rightarrow (E)\bullet\}$, 执行归约: 弹出 3 个符号 $)$, E , $($ 和 3 个状态 9, 8, 3。栈顶变为状态 0。goto(I_0, E) = I_2 , 压入 E 和状态 2。

3.5 G2 的状态 I_8 冲突分析 (4 分)

G2 的 LR(0) 项目集状态 I_8 :

$T \rightarrow b \bullet$ (归约项目: 按 $T \rightarrow b$ 归约)

$T \rightarrow b \bullet E$ (移进项目: 期望移进 E 的 FIRST 集符号)

$E \rightarrow \bullet(E)$

$E \rightarrow \bullet a$

LR(0) 中是否存在冲突?

是的, 存在移进-归约冲突。 $T \rightarrow b \bullet$ 要求归约, 而 $T \rightarrow b \bullet E$ (以及 E 的闭包) 要求移进 (或 a)。LR(0) 不考虑前瞻符, 无法区分应该归约还是移进。

SLR(1) 能否解决?

可以解决。SLR(1) 在决定是否归约 $T \rightarrow b$ 时, 只在输入符号属于 FOLLOW(T) 时才归约。

计算 FOLLOW(T) (G2 中):

- 从 $S \rightarrow E + T$: T 后面是 $\$$ (或 FOLLOW(S)), 故 $\$ \in \text{FOLLOW}(T)$
- 从 $T \rightarrow bE$: T 是左部, 不贡献 FOLLOW
- T 不出现在其他产生式的右部

故 FOLLOW(T) = $\{\$ \}$ 。

在 SLR(1) 中, 状态 I_8 的归约动作 $T \rightarrow b$ 仅在输入为 $\$$ 时才执行。而移进动作仅在输入为 (或 a ($\in \text{FIRST}(E)$)) 时才执行。由于 $\{\$ \} \cap \{ (, a \} = \emptyset$, 没有交集, 冲突被消除。

结论: SLR(1) 可以解决该冲突。

3.6 LR(1) 的改进原理与闭包计算方法 (4 分)

一、LR(1) 项目是什么?

LR(1) 项目形如 $[A \rightarrow \alpha \bullet \beta, a]$, 包含两部分:

- **核心部分** $A \rightarrow \alpha \bullet \beta$: 与 LR(0) 项目相同, 表示已看到 α 、期望看到 β
- **前瞻符 (lookahead) a** : 一个终结符, 表示只有当当前输入符号是 a 时, 才允许按此项目归约 (当 $\beta = \varepsilon$ 时)

举例: $[E \rightarrow \bullet(E), +]$ 的含义是“我们在期望规约一个 E , 而且这个 E 的右侧必须紧跟着一个 $+$ ”。

二、LR(1) 为什么比 SLR(1) 更精确？

SLR(1) 对产生式 $A \rightarrow \gamma$ 的所有出现统一使用 FOLLOW(A) 来决定何时归约。但 A 可能在文法的不同上下文中出现，不同上下文中 A 后面能跟的符号是不同的。SLR(1) 把所有这些可能混在一起，导致不精确。

LR(1) 给每个项目单独附加一个精确的前瞻符。同一个产生式 $A \rightarrow \gamma$ 在不同上下文中可能带不同的前瞻符，从而只在正确的上下文中归约。

三、闭包计算的通用方法

LR(1) 闭包的计算是 LR(1) 最核心的算法。从初始项目集出发，反复应用以下规则直到不再产生新项目：

闭包扩展规则：

若状态中存在项目 $[A \rightarrow \alpha \bullet B\beta, a]$ (即 $\bullet$ 后面是一个非终结符 B)，则对 B 的每个产生式 $B \rightarrow \gamma$ ，向该状态添加项目：

$$[B \rightarrow \bullet \gamma, b], \quad \text{其中 } b \in \text{FIRST}(\beta a)$$

关键问题：为什么前瞻符是 $\text{FIRST}(\beta a)$ ？

在项目 $[A \rightarrow \alpha \bullet B\beta, a]$ 中：

- 我们期望接下来看到 B (由产生式 $B \rightarrow \gamma$ 推导)
- B 推导完毕后，后面紧跟着的是 β (如果 $\beta \neq \epsilon$) 或 a (如果 β 能推导出 ϵ)
- 因此，当我们最终归约 $B \rightarrow \gamma$ 时，当前输入符号必须是 $\text{FIRST}(\beta a)$ 中的某个符号
- 这个符号就被设置为新项目 $[B \rightarrow \bullet \gamma, b]$ 的前瞻符

四、以 G2 为例，逐步推导初始状态 I_0

G2 拓广后：

$$(0) S' \rightarrow S$$

$$(1) S \rightarrow E + T$$

$$(2) E \rightarrow (E)$$

$$(3) E \rightarrow a$$

$$(4) T \rightarrow b$$

$$(5) T \rightarrow bE$$

第 1 步：放入种子项目。 LR(1) DFA 的初始状态始终从 $[S' \rightarrow \bullet S, \$]$ 开始，表示“期望看到 S ，然后文件结束”。

$$I_0 = \{ [S' \rightarrow \bullet S, \$] \}$$

第 2 步：闭包扩展（第 1 轮）。检查 $[S' \rightarrow \bullet S, \$]$ ： $\bullet$ 后面是 S （非终结符），触发扩展。
对 S 的产生式 $S \rightarrow E + T$ ，计算前瞻符：

- 在 $S' \rightarrow \bullet S$ 中， $B = S$ ， $\beta = \varepsilon$ （ S 后面什么都没有）， $a = \$$
- 所以前瞻符 $b \in \text{FIRST}(\varepsilon \cdot \$) = \text{FIRST}(\$) = \{\$ \}$

添加： $[S \rightarrow \bullet E + T, \$]$

现在 $I_0 = \{ [S' \rightarrow \bullet S, \$], [S \rightarrow \bullet E + T, \$] \}$

第 3 步：闭包扩展（第 2 轮）。检查新项目 $[S \rightarrow \bullet E + T, \$]$ ： $\bullet$ 后面是 E （非终结符），触发扩展。

对 E 的产生式 $E \rightarrow (E)$ 和 $E \rightarrow a$ ，计算前瞻符：

- 在 $S \rightarrow \bullet E + T$ 中， $B = E$ ， $\beta = +T$ ， $a = \$$
- 前瞻符 $b \in \text{FIRST}(+T \cdot \$) = \text{FIRST}(+T) = \{+\}$
- 解释：归约出 E 之后，后面应该跟着 $+T\$$ ，所以紧跟着的第一个符号必须是 $+$

添加： $[E \rightarrow \bullet(E), +]$ 和 $[E \rightarrow \bullet a, +]$

第 4 步：检查是否还有新项目需扩展。新添加的项目 $[E \rightarrow \bullet(E), +]$ 的 $\bullet$ 后面是 $($ （终结符），不触发扩展。 $[E \rightarrow \bullet a, +]$ 的 $\bullet$ 后面是 a （终结符），也不触发扩展。

闭包计算结束。最终：

$$I_0 = \left\{ \begin{array}{l} [S' \rightarrow \bullet S, \$], \\ [S \rightarrow \bullet E + T, \$], \\ [E \rightarrow \bullet(E), +], \\ [E \rightarrow \bullet a, +] \end{array} \right\}$$

五、前瞻符的意义——一个具体对比

假设后续分析中，从 I_0 经移进和归约到达了含 $[E \rightarrow a\bullet, +]$ 的状态。此时：

- **SLR(1) 的做法：**查 $\text{FOLLOW}(E)$ 。在 G_2 中， E 出现在 $S \rightarrow E + T$ （后跟 $+$ ）和 $E \rightarrow (E)$ （后跟 $)$ ）中，故 $\text{FOLLOW}(E) = \{+,)\}$ 。SLR(1) 在输入为 $+$ 或 $)$ 时都按 $E \rightarrow a$ 归约。但如果输入是 $)$ ，归约 $E \rightarrow a$ 可能是错误的——因为我们当前所在的上下文可能不允许 E 后面跟 $)$ 。
- **LR(1) 的做法：**直接看前瞻符。这个项目的 lookahead 是 $+$ ，所以只在输入为 $+$ 时才归约。输入为 $)$ 时不会错误归约。

这就是 LR(1) 比 SLR(1) 强大的本质：每个项目带着自己精确的”触发条件”，而不是用一个笼统的 FOLLOW 集。

六、通用闭包算法（伪代码）

```

function closure( $I$ ):
  repeat:
    for each item  $[A \rightarrow \alpha \bullet B\beta, a]$  in  $I$ :
      for each production  $B \rightarrow \gamma$ :
        for each terminal  $b \in \text{FIRST}(\beta a)$ :
          add  $[B \rightarrow \bullet\gamma, b]$  to  $I$ 
    until  $I$  no longer changes
  return  $I$ 
  
```

其中 $\text{FIRST}(\beta a)$ 的计算规则与标准 FIRST 集计算一致：

- 若 $\beta = \varepsilon$ ，则 $\text{FIRST}(\varepsilon a) = \{a\}$ ：前瞻符直接继承自外层项目的 a
- 若 β 非空，则依次取 β 中各符号的 FIRST 集，直到遇到不能推导 ε 的符号；若 β 整体可推导 ε ，最后还要并上 $\{a\}$

七、与 SLR(1) 的对比总结

SLR(1)	LR(1)
归约条件：输入 $\in \text{FOLLOW}(A)$ ，对所有 $A \rightarrow \gamma$ 的项目统一使用	归约条件：输入 $= a$ ，其中 a 是 该项目 的前瞻符
前瞻符来源：FOLLOW(A) 从整个文法的产生式中统计	前瞻符来源：通过闭包运算从上下文逐步计算，精确到每个出现位置
状态数与 LR(0) 相同	状态数更多（同一个 LR(0) 核心可能因前瞻符不同而分裂为多个 LR(1) 状态）
能力较弱，部分文法无法处理	能力最强，LR(1) 文法严格包含 SLR(1) 文法